

TypeScript Fundamentals - Part 3

Functions & Type Annotations

Type Annotations and Type Inference

TypeScript provides two ways to assign types to variables: explicit type annotations and automatic type inference. Understanding when to use each approach is crucial for writing clean, maintainable TypeScript code.

Explicit Type Annotations

Explicit type annotations are when you manually specify the type of a variable, parameter, or return value. You write the type after a colon following the variable name.

```
// Explicit type annotation for variables
let username: string = "Rajesh";
let age: number = 30;
let isStudent: boolean = false;

console.log("Username:", username);
console.log("Age:", age);
console.log("Is Student:", isStudent);

// Explicit type annotation for arrays
let scores: number[] = [85, 90, 78];
let names: string[] = ["Amit", "Priya", "Neha"];

console.log("Scores:", scores);
console.log("Names:", names);
```

Output:

```
Username: Rajesh
Age: 30
Is Student: false
Scores: [ 85, 90, 78 ]
Names: [ 'Amit', 'Priya', 'Neha' ]
```

Explicit annotations make your intent clear to other developers and to the TypeScript compiler. They serve as documentation and provide immediate feedback if you try to assign the wrong type.

Type Inference

Type inference is TypeScript's ability to automatically determine the type of a variable based on the value assigned to it. When you initialize a variable without specifying a type, TypeScript infers the type from the initial value.

```
// Type inference - TypeScript automatically determines the type
let city = "Mumbai"; // TypeScript infers type as string
let population = 20000000; // TypeScript infers type as number
let isCapital = false; // TypeScript infers type as boolean

console.log("City:", city);
console.log("Population:", population);
console.log("Is Capital:", isCapital);

// TypeScript infers array types
let numbers = [1, 2, 3, 4, 5]; // Inferred as number[]
let fruits = ["Apple", "Banana", "Orange"]; // Inferred as string[]

console.log("Numbers:", numbers);
console.log("Fruits:", fruits);
```

Output:

```
City: Mumbai
Population: 20000000
Is Capital: false
Numbers: [ 1, 2, 3, 4, 5 ]
Fruits: [ 'Apple', 'Banana', 'Orange' ]
```

Even though we didn't specify types, TypeScript knows that `city` is a string, `population` is a number, and so on. If you try to assign a value of the wrong type later, TypeScript will catch the error.

```
let country = "India"; // Inferred as string
// country = 123; // ✗ Error: Type 'number' is not assignable to type 'string'
country = "Bharat"; // ✓ Correct - string assignment is allowed

console.log("Country:", country);
```

Output:

```
Country: Bharat
```

When to Use Explicit vs Implicit Typing

Choosing between explicit annotations and type inference depends on the situation. Here are some guidelines:

Use Explicit Type Annotations when:

- Declaring a variable without immediately initializing it
- The inferred type is not specific enough for your needs
- You want to make the code more readable and self-documenting
- Working with function parameters and return types (always recommended)

```
// Variable declared without initialization - explicit type needed
let email: string;
email = "user@example.com";
console.log("Email:", email);
```

```
// Making intent clear with explicit type
let userId: string | number = "USER123"; // Could be string or number
console.log("User ID:", userId);

// Function with explicit parameter and return types
function calculateTotal(price: number, quantity: number): number {
    return price * quantity;
}

console.log("Total:", calculateTotal(500, 3));
```

Output:

```
Email: user@example.com
User ID: USER123
Total: 1500
```

Use Type Inference when:

- The initial value makes the type obvious
- You want cleaner, more concise code
- The inferred type is exactly what you need

```
// Type inference works perfectly here
let product = "Laptop";
let price = 45000;
let available = true;

console.log(`Product: ${product}, Price: ${price}, Available: ${available}`);
```

Output:

```
Product: Laptop, Price: 45000, Available: true
```

Best Practices for Type Annotations

Here are some best practices to follow when working with types in TypeScript:

```
// ☒ Good: Explicit types for function parameters and return values
function greetUser(name: string, age: number): string {
    return `Hello, ${name}! You are ${age} years old.`;
}

// ☒ Good: Let TypeScript infer obvious types
let message = greetUser("Suresh", 28);

// ☒ Good: Explicit type when declaring without initializing
let response: string;
response = "Success";

// ☒ Good: Union types when value can be multiple types
let id: string | number = 12345;

console.log(message);
console.log("Response:", response);
console.log("ID:", id);
```

Output:

```
Hello, Suresh! You are 28 years old.
Response: Success
ID: 12345
```

Functions in TypeScript

Functions are the building blocks of any application. TypeScript enhances JavaScript functions with type safety, making them more reliable and easier to understand.

Function Type Annotations

In TypeScript, you can specify types for function parameters and return values. This ensures that functions are called with the correct arguments and return the expected type.

```
// Basic function with type annotations
function add(a: number, b: number): number {
    return a + b;
}

function greet(name: string): string {
    return `Welcome, ${name}!`;
}

function isEligible(age: number): boolean {
    return age >= 18;
}

console.log("Add:", add(10, 20));
console.log("Greet:", greet("Kavya"));
console.log("Is Eligible:", isEligible(25));
console.log("Is Eligible:", isEligible(15));
```

Output:

```
Add: 30
Greet: Welcome, Kavya!
Is Eligible: true
Is Eligible: false
```

The syntax is straightforward: parameter types are specified after each parameter name, and the return type comes after the parameter list, before the function body.

Parameter Type Annotations

Every parameter in a TypeScript function can have a type annotation. This prevents passing incorrect data to functions.

```
// Function with multiple typed parameters
function createUser(name: string, age: number, email: string, isActive: boolean): void {
    console.log("Creating user...");
    console.log(`Name: ${name}`);
    console.log(`Age: ${age}`);
    console.log(`Email: ${email}`);
}
```

```
    console.log(`Active: ${isActive}`);
  }

  createUser("Arun", 32, "arun@example.com", true);

  // Function with array and object parameters
  function processOrder(items: string[], total: number, customer: { name: string; phone:
string }): void {
    console.log("Processing order for:", customer.name);
    console.log("Phone:", customer.phone);
    console.log("Items:", items);
    console.log("Total:", total);
  }

  processOrder(
    ["Laptop", "Mouse", "Keyboard"],
    47500,
    { name: "Deepak", phone: "9876543210" }
  );
```

Output:

```
Creating user...
Name: Arun
Age: 32
Email: arun@example.com
Active: true
Processing order for: Deepak
Phone: 9876543210
Items: [ 'Laptop', 'Mouse', 'Keyboard' ]
Total: 47500
```

Return Type Annotations

Specifying return types helps catch errors where a function might return an unexpected value. While TypeScript can infer return types, explicitly stating them is considered a best practice.

```
// Explicit return type annotations
function multiply(x: number, y: number): number {
    return x * y;
}

function getFullName(firstName: string, lastName: string): string {
    return `${firstName} ${lastName}`;
}

function hasDiscount(totalPurchase: number): boolean {
    return totalPurchase > 5000;
}

console.log("Multiply:", multiply(6, 7));
console.log("Full Name:", getFullName("Ravi", "Kumar"));
console.log("Has Discount:", hasDiscount(7500));

// Function returning an object
function createProduct(name: string, price: number): { name: string; price: number; id:
number } {
    return {
        name: name,
        price: price,
        id: Math.floor(Math.random() * 1000)
    };
}

let product = createProduct("Phone", 15000);
console.log("Product:", product);
```

Output:

```
Multiply: 42
Full Name: Ravi Kumar
Has Discount: true
Product: { name: 'Phone', price: 15000, id: 734 }
```

The id will be different each time due to the random number generation.

JavaScript Parameters vs TypeScript Parameters

Understanding the difference between JavaScript and TypeScript parameters is essential. In JavaScript, parameters are not type-checked, which can lead to runtime errors. TypeScript adds type safety to prevent these issues.

JavaScript Parameters (Without Types)

```
// JavaScript function - no type checking
function calculateArea(length, width) {
    return length * width;
}

console.log("Area 1:", calculateArea(10, 5));
console.log("Area 2:", calculateArea("10", "5")); // Returns "105" (string concatenation!)
console.log("Area 3:", calculateArea(10)); // Returns NaN (width is undefined)
```

Output:

```
Area 1: 50
Area 2: 105
Area 3: NaN
```

JavaScript allows these calls, but they produce unexpected results.

TypeScript Parameters (With Types)

```
// TypeScript function - type-safe parameters
function calculateAreaTS(length: number, width: number): number {
    return length * width;
}

console.log("Area:", calculateAreaTS(10, 5));

// These would cause compilation errors:
// calculateAreaTS("10", "5"); // ✗ Error: Argument of type 'string' is not assignable
// calculateAreaTS(10); // ✗ Error: Expected 2 arguments, but got 1
```

Output:

Area: 50

TypeScript prevents incorrect function calls at compile time.

Optional Parameters

Optional parameters allow you to call a function without providing all arguments. In TypeScript, you mark a parameter as optional by adding a question mark (?) after the parameter name.

```
// Function with optional parameter
function greetPerson(name: string, age?: number): string {
    if (age !== undefined) {
        return `Hello, ${name}! You are ${age} years old.`;
    } else {
        return `Hello, ${name}!`;
    }
}

console.log(greetPerson("Anjali", 25));
console.log(greetPerson("Vikram")); // age is optional

// Multiple optional parameters
function createAccount(username: string, email: string, phone?: string, address?: string): void
{
    console.log("Username:", username);
    console.log("Email:", email);
    if (phone) console.log("Phone:", phone);
    if (address) console.log("Address:", address);
}

createAccount("ravi123", "ravi@example.com");
console.log("---");
createAccount("sneha456", "sneha@example.com", "9876543210");
console.log("---");
createAccount("amit789", "amit@example.com", "8765432109", "Mumbai, India");
```

Output:

Hello, Anjali! You are 25 years old.

Hello, Vikram!

Username: ravi123

Email: ravi@example.com

Username: sneha456

Email: sneha@example.com

Phone: 9876543210

Username: amit789

Email: amit@example.com

Phone: 8765432109

Address: Mumbai, India

Note:

Important: Optional parameters must come after required parameters. You cannot have a required parameter after an optional one.

```
// ✗ Wrong: Required parameter after optional
// function wrong(name?: string, age: number): void { }

// ✓ Correct: Optional parameters at the end
function correct(age: number, name?: string): void {
  console.log(`Age: ${age}, Name: ${name || "Not provided"}`);
}

correct(30);
correct(25, "Riya");
```

Output:

Age: 30, Name: Not provided

Age: 25, Name: Riya

Default Parameters

Default parameters allow you to specify a default value for a parameter. If the caller doesn't provide that argument, the default value is used.

```
// Function with default parameters
function calculatePrice(price: number, tax: number = 0.18, discount: number = 0): number {
    const priceWithTax = price + (price * tax);
    const finalPrice = priceWithTax - discount;
    return finalPrice;
}

console.log("Price with default tax:", calculatePrice(1000));
console.log("Price with custom tax:", calculatePrice(1000, 0.12));
console.log("Price with tax and discount:", calculatePrice(1000, 0.18, 200));

// Default parameters with strings
function sendMessage(message: string, recipient: string = "Admin", priority: string = "normal"): void {
    console.log(`To: ${recipient}`);
    console.log(`Priority: ${priority}`);
    console.log(`Message: ${message}`);
    console.log("---");
}

sendMessage("System update completed");
sendMessage("Error occurred", "IT Team");
sendMessage("Critical issue", "Manager", "high");
```

Output:

```
Price with default tax: 1180
Price with custom tax: 1120
Price with tax and discount: 980
To: Admin
Priority: normal
Message: System update completed
---
To: IT Team
Priority: normal
```

Message: Error occurred

To: Manager

Priority: high

Message: Critical issue

Using Default Parameters Before Required Parameters

While it's uncommon, you can have default parameters before required ones. However, to skip the default parameter, you must explicitly pass `undefined`.

```
// Default parameter before required parameter
function createUser(role: string = "user", name: string, id: number): void {
    console.log(`ID: ${id}, Name: ${name}, Role: ${role}`);
}

// To use the default, pass undefined explicitly
createUser(undefined, "Rajesh", 101);

// Or provide a custom value
createUser("admin", "Priya", 102);
```

Output:

ID: 101, Name: Rajesh, Role: user

ID: 102, Name: Priya, Role: admin

Note:

Best Practice: Keep default parameters at the end of the parameter list to avoid having to pass `undefined` explicitly. This makes function calls cleaner and more intuitive.

Rest Parameters

Rest parameters allow a function to accept an indefinite number of arguments as an array. You use the spread operator (`...`) before the parameter name.

```
// Function with rest parameters
function sumAll(...numbers: number[]): number {
    let total = 0;
    for (let num of numbers) {
        total += num;
    }
    return total;
}

console.log("Sum of 2 numbers:", sumAll(10, 20));
console.log("Sum of 5 numbers:", sumAll(5, 10, 15, 20, 25));
console.log("Sum of 8 numbers:", sumAll(1, 2, 3, 4, 5, 6, 7, 8));

// Rest parameters with other parameters
function createTeam(teamName: string, leader: string, ...members: string[]): void {
    console.log(`Team: ${teamName}`);
    console.log(`Leader: ${leader}`);
    console.log(`Members: ${members.join(", ")}`);
    console.log(`Total team size: ${members.length + 1}`);
    console.log("---");
}

createTeam("Development", "Arjun", "Sneha", "Raj", "Meera");
createTeam("Testing", "Pooja", "Amit", "Vivek");
```

Output:

```
Sum of 2 numbers: 30
Sum of 5 numbers: 75
Sum of 8 numbers: 36
Team: Development
Leader: Arjun
Members: Sneha, Raj, Meera
Total team size: 4
---
```

```
Team: Testing
Leader: Pooja
Members: Amit, Vivek
Total team size: 3
---
```

Rest Parameters with Different Types

```
// Rest parameters can work with different types
function logMessages(...messages: string[]): void {
    messages.forEach((msg, index) => {
        console.log(`Message ${index + 1}: ${msg}`);
    });
}

logMessages("Hello", "Welcome", "TypeScript is great!");

// Combining multiple parameter types
function formatData(separator: string, ...items: (string | number)[]): string {
    return items.join(separator);
}

console.log(formatData(" - ", "Name", "Age", 25, "City", "Mumbai"));
```

Output:

```
Message 1: Hello
Message 2: Welcome
Message 3: TypeScript is great!
Name - Age - 25 - City - Mumbai
```

Note:

Important: Rest parameters must be the last parameter in the function. You can only have one rest parameter per function.

Passing All Parameter Types in a Single Function

TypeScript allows you to combine required parameters, optional parameters, default parameters, and rest parameters in a single function. However, they must follow a specific order.

```
// Combining all parameter types
// Order: required → optional → default → rest
function processOrder(
  orderId: number,           // Required parameter
  customerName: string,      // Required parameter
  email?: string,            // Optional parameter
  priority: string = "normal", // Default parameter
  ...items: string[]         // Rest parameter
): void {
  console.log("Order ID:", orderId);
  console.log("Customer:", customerName);
  if (email) console.log("Email:", email);
  console.log("Priority:", priority);
  console.log("Items:", items.join(", "));
  console.log("---");
}

// Different ways to call the function
processOrder(101, "Rahul", undefined, undefined, "Laptop", "Mouse");
processOrder(102, "Priya", "priya@example.com", "high", "Phone", "Charger", "Case");
processOrder(103, "Amit", "amit@example.com", undefined, "Tablet");
processOrder(104, "Neha");
```

Output:

```
Order ID: 101
Customer: Rahul
Priority: normal
Items: Laptop, Mouse
---
Order ID: 102
Customer: Priya
Email: priya@example.com
Priority: high
Items: Phone, Charger, Case
```

```
---  
Order ID: 103  
Customer: Amit  
Email: amit@example.com  
Priority: normal  
Items: Tablet  
---  
Order ID: 104  
Customer: Neha  
Priority: normal  
Items:  
---
```

Function Overloading

Function overloading allows you to define multiple function signatures for the same function name. TypeScript will choose the correct signature based on the arguments provided.

```
// Function overload signatures  
function getValue(id: number): string;  
function getValue(name: string): number;  
  
// Implementation signature  
function getValue(input: number | string): string | number {  
  if (typeof input === "number") {  
    return `ID-${input}`;  
  } else {  
    return input.length;  
  }  
}  
  
console.log("Get by number:", getValue(123));  
console.log("Get by string:", getValue("TypeScript"));  
  
// Practical example: formatting dates  
function formatDate(date: Date): string;  
function formatDate(day: number, month: number, year: number): string;  
  
function formatDate(dayOrDate: Date | number, month?: number, year?: number): string {
```

```
if (dayOrDate instanceof Date) {
    return dayOrDate.toString();
} else {
    return `${dayOrDate}/${month}/${year}`;
}

console.log("Format 1:", formatDate(new Date()));
console.log("Format 2:", formatDate(15, 8, 2024));
```

Output:

```
Get by number: ID-123
Get by string: 10
Format 1: Tue Jan 21 2025
Format 2: 15/8/2024
```

Void and Never Return Types

We've already seen `void` and `never` types. Let's explore them specifically in the context of function return types.

Void Return Type

Functions that don't return a value have a `void` return type. These functions are called for their side effects, not for their return value.

```
// Void functions
function logToConsole(message: string): void {
    console.log(`[LOG]: ${message}`);
}

function updateUI(element: string, value: string): void {
    console.log(`Updating ${element} with value: ${value}`);
    // In a real application, this would update the DOM
}

function saveToDatabase(data: any): void {
    console.log("Saving data to database:", data);
    // No return value
```

```
}

logToConsole("Application started");
updateUI("header", "Welcome to TypeScript");
saveToDatabase({ user: "Rahul", age: 28 });
```

Output:

```
[LOG]: Application started
Updating header with value: Welcome to TypeScript
Saving data to database: { user: 'Rahul', age: 28 }
```

While void functions don't explicitly return a value, they can technically return `undefined`. However, TypeScript treats them as not returning anything meaningful.

```
function doSomething(): void {
    console.log("Doing something...");
    // These are allowed in void functions:
    return;
    // return undefined;
}

// But this is NOT allowed:
// function wrong(): void {
//     return 5; // ✗ Error: Type 'number' is not assignable to type 'void'
// }

doSomething();
```

Output:

```
Doing something...
```

Never Return Type

The `never` type represents functions that never return. This happens in two scenarios: functions that throw errors and functions that run indefinitely.

```
// Function that throws an error - never returns normally
function throwError(errorMessage: string): never {
    throw new Error(errorMessage);
}

// Function that runs indefinitely - never returns
function infiniteLoop(): never {
    while (true) {
        console.log("Running...");
        // This loop never ends
        break; // Remove this in actual infinite loop
    }
    throw new Error("Should never reach here");
}

// Practical use: type-safe error handling
function processStatus(status: "success" | "error" | "pending"): string {
    switch (status) {
        case "success":
            return "Operation completed successfully";
        case "error":
            return "Operation failed";
        case "pending":
            return "Operation in progress";
        default:
            // This should never happen if all cases are covered
            const exhaustiveCheck: never = status;
            throw new Error(`Unhandled status: ${exhaustiveCheck}`);
    }
}

console.log(processStatus("success"));
console.log(processStatus("error"));

// Uncommenting this will throw an error:
// throwError("Something went wrong!");
```

Output:

```
Running...
Operation completed successfully
Operation failed
```

Note:

Understanding Never: The `never` type is different from `void`. A `void` function completes and returns `undefined`, while a `never` function doesn't complete normally at all.

Arrow Functions in TypeScript

Arrow functions (also called lambda functions) provide a concise syntax for writing functions. TypeScript supports arrow functions with full type safety.

```
// Traditional function
function addTraditional(a: number, b: number): number {
    return a + b;
}

// Arrow function with explicit return
const addArrow = (a: number, b: number): number => {
    return a + b;
};

// Arrow function with implicit return
const addShort = (a: number, b: number): number => a + b;

console.log("Traditional:", addTraditional(10, 20));
console.log("Arrow with return:", addArrow(15, 25));
console.log("Arrow short:", addShort(30, 40));

// Arrow functions with different parameter counts
const greet = (name: string): string => `Hello, ${name}!`;
const square = (n: number): number => n * n;
const getPI = (): number => 3.14159;

console.log(greet("Anjali"));
console.log("Square of 7:", square(7));
console.log("PI value:", getPI());
```

Output:

```
Traditional: 30
Arrow with return: 40
Arrow short: 70
Hello, Anjali!
Square of 7: 49
PI value: 3.14159
```

Arrow Functions with Type Inference

TypeScript can infer the return type of arrow functions based on the return statement, but it's still good practice to specify it explicitly.

```
// Type inference with arrow functions
const multiply = (x: number, y: number) => x * y; // Return type inferred as number
const isAdult = (age: number) => age >= 18; // Return type inferred as boolean
const getName = (firstName: string, lastName: string) => `${firstName} ${lastName}`; //
Inferred as string

console.log("Multiply:", multiply(6, 8));
console.log("Is Adult:", isAdult(20));
console.log("Name:", getName("Ravi", "Sharma"));

// Arrow function with array methods
const numbers: number[] = [1, 2, 3, 4, 5];

const doubled = numbers.map((n: number): number => n * 2);
const evenNumbers = numbers.filter((n: number): boolean => n % 2 === 0);
const sum = numbers.reduce((acc: number, n: number): number => acc + n, 0);

console.log("Doubled:", doubled);
console.log("Even numbers:", evenNumbers);
console.log("Sum:", sum);
```

Output:

```
Multiply: 48
Is Adult: true
Name: Ravi Sharma
```

```
Doubled: [ 2, 4, 6, 8, 10 ]  
Even numbers: [ 2, 4 ]  
Sum: 15
```

Function Types as Variables

In TypeScript, you can define the type of a function and use it to declare variables that hold functions. This is useful when you want to pass functions as arguments or return them from other functions.

```
// Define a function type  
type MathOperation = (a: number, b: number) => number;  
  
// Variables that hold functions matching this type  
const add: MathOperation = (a, b) => a + b;  
const subtract: MathOperation = (a, b) => a - b;  
const multiply: MathOperation = (a, b) => a * b;  
const divide: MathOperation = (a, b) => b !== 0 ? a / b : 0;  
  
console.log("Add:", add(10, 5));  
console.log("Subtract:", subtract(10, 5));  
console.log("Multiply:", multiply(10, 5));  
console.log("Divide:", divide(10, 5));  
  
// Function that takes another function as parameter  
function calculate(x: number, y: number, operation: MathOperation): number {  
    return operation(x, y);  
}  
  
console.log("Calculate with add:", calculate(20, 10, add));  
console.log("Calculate with multiply:", calculate(20, 10, multiply));
```

Output:

```
Add: 15  
Subtract: 5  
Multiply: 50
```

```
Divide: 2
Calculate with add: 30
Calculate with multiply: 200
```

Callbacks and Higher-Order Functions

Higher-order functions are functions that take other functions as parameters or return functions. TypeScript ensures type safety for these patterns.

```
// Callback function type
type Callback = (result: string) => void;

function fetchData(url: string, callback: Callback): void {
    console.log(`Fetching data from ${url}...`);
    // Simulate async operation
    const data = `Data from ${url}`;
    callback(data);
}

fetchData("https://api.example.com/users", (result) => {
    console.log("Received:", result);
});

// Function that returns a function
type Multiplier = (n: number) => number;

function createMultiplier(factor: number): Multiplier {
    return (n: number): number => n * factor;
}

const double = createMultiplier(2);
const triple = createMultiplier(3);

console.log("Double 5:", double(5));
console.log("Triple 5:", triple(5));

// Practical example: event handlers
type EventHandler = (event: string) => void;

function addEventListener(eventName: string, handler: EventHandler): void {
    console.log(`Event listener added for: ${eventName}`);
    handler(`${eventName} triggered`);
}
```

```
addEventListener("click", (event) => {  
  console.log("Handling event:", event);  
});
```

Output:

```
Fetching data from https://api.example.com/users...  
Received: Data from https://api.example.com/users  
Double 5: 10  
Triple 5: 15  
Event listener added for: click  
Handling event: click triggered
```

Practical Examples: Combining Concepts

Let's combine everything we've learned about functions into practical, real-world examples.

Example 1: User Registration System

```
// Type definitions  
type User = {  
  id: number;  
  username: string;  
  email: string;  
  role: string;  
};  
  
type ValidationResult = {  
  isValid: boolean;  
  errors: string[];  
};  
  
// Validate email format  
function validateEmail(email: string): boolean {  
  return email.includes("@") && email.includes(".");  
}  
  
// Validate user data  
function validateUser(username: string, email: string, age?: number): ValidationResult {
```

```
const errors: string[] = [];

if (username.length < 3) {
  errors.push("Username must be at least 3 characters");
}

if (!validateEmail(email)) {
  errors.push("Invalid email format");
}

if (age !== undefined && age < 18) {
  errors.push("User must be at least 18 years old");
}

return {
  isValid: errors.length === 0,
  errors: errors
};
}

// Register user with default role
function registerUser(
  username: string,
  email: string,
  role: string = "user",
  age?: number
): User | null {
  const validation = validateUser(username, email, age);

  if (!validation.isValid) {
    console.log("Registration failed:");
    validation.errors.forEach(error => console.log(` - ${error}`));
    return null;
  }

  const user: User = {
    id: Math.floor(Math.random() * 10000),
    username,
    email,
    role
  };

  console.log("User registered successfully:", user);
  return user;
}

// Test the registration system
```

```
registerUser("raj", "raj@example.com");
console.log("---");
registerUser("Rajesh", "rajesh@example.com", "admin");
console.log("---");
registerUser("Priya", "priyaexample", "user", 25);
console.log("---");
registerUser("Amit", "amit@example.com", undefined, 16);
```

Output:

```
User registered successfully: { id: 7423, username: 'raj', email: 'raj@example.com', role:
'user' }
---
User registered successfully: { id: 3891, username: 'Rajesh', email: 'rajesh@example.com',
role: 'admin' }
---
Registration failed:
- Invalid email format
---
Registration failed:
- User must be at least 18 years old

IDs will vary due to random generation.
```

Example 2: Shopping Cart Calculator

```
// Type definitions
type Product = {
  name: string;
  price: number;
  quantity: number;
};

type DiscountFunction = (total: number) => number;

// Calculate subtotal
function calculateSubtotal(...products: Product[]): number {
  return products.reduce((total, product) => {
```

```

    return total + (product.price * product.quantity);
  }, 0);
}

// Apply discount based on total
function createDiscountCalculator(threshold: number, discountPercent: number): DiscountFunction
{
  return (total: number): number => {
    if (total >= threshold) {
      return total * (discountPercent / 100);
    }
    return 0;
  };
}

// Calculate final price with tax and discount
function calculateFinalPrice(
  subtotal: number,
  taxRate: number = 0.18,
  discountCalculator?: DiscountFunction
): { subtotal: number; tax: number; discount: number; total: number } {
  const tax = subtotal * taxRate;
  const discount = discountCalculator ? discountCalculator(subtotal) : 0;
  const total = subtotal + tax - discount;

  return { subtotal, tax, discount, total };
}

// Test the shopping cart
const products: Product[] = [
  { name: "Laptop", price: 50000, quantity: 1 },
  { name: "Mouse", price: 500, quantity: 2 },
  { name: "Keyboard", price: 1500, quantity: 1 }
];

const subtotal = calculateSubtotal(...products);
console.log("Subtotal:", subtotal);

const bulkDiscount = createDiscountCalculator(50000, 10); // 10% off above 50000
const finalPrice = calculateFinalPrice(subtotal, 0.18, bulkDiscount);

console.log("Tax:", finalPrice.tax);
console.log("Discount:", finalPrice.discount);
console.log("Final Total:", finalPrice.total);
console.log("---");

// Without discount

```

```
const smallOrder: Product[] = [
  { name: "Pen", price: 10, quantity: 5 }
];

const smallSubtotal = calculateSubtotal(...smallOrder);
const smallFinalPrice = calculateFinalPrice(smallSubtotal);

console.log("Small Order Subtotal:", smallSubtotal);
console.log("Small Order Total:", smallFinalPrice.total);
```

Output:

Subtotal: 52500

Tax: 9450

Discount: 5250

Final Total: 56700

Small Order Subtotal: 50

Small Order Total: 59

Summary

In this section, we explored TypeScript functions in depth. You learned about type annotations and type inference, understanding when to use explicit types versus letting TypeScript infer them. We covered all aspects of function parameters including required, optional, default, and rest parameters, along with their proper ordering.

We examined function return types, including `void` for functions without return values and `never` for functions that never return. You learned about arrow functions, function overloading, and how to use functions as types and pass them as arguments.

The key takeaways are: always type your function parameters and return values, use optional and default parameters to make functions more flexible, leverage rest parameters for variable arguments, and understand the difference between `void` and `never` return types.



In the next section, we'll explore interfaces and classes, which will help you structure larger applications using object-oriented programming principles in TypeScript.

